Go2 SDK Development Guide

Search article titles

About Go2
Remote Controller
SDK Concepts 3
Application Development 7
Software Interface Services 19
Motion Services Interface V2.0
DDS Services Interface
Avoidance Services Interface
VuiClient Interface
RobotStateClient Interface
Motion Switcher Service Interface
Sports Services Interface
AI motion Services interface
Basic Services Interface
LiDAR Services Interface
Multimedia Services Interface
UWB Services Interface
Fault Services Interface
SLAM and Navigation Services Interfa
Aruco Recharge Service Interface
Odometer Services Interface
D1 Mechanical Arm Services Interface
ROS2 Services Interface
Python Services Interface
Case reference 8
Debugging Specification
FAQ

Fault Services Interface

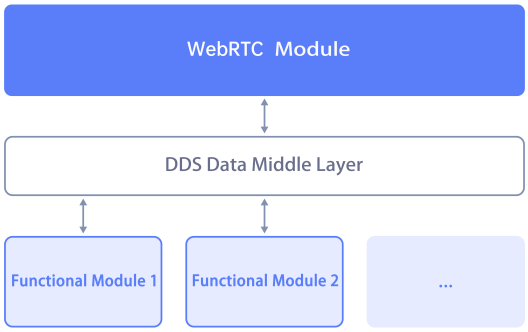
Last Updated On : 2023-10-19 16:27:07

This page contains content

Introduction
Error code

Introduction

Fault information is distributed across different DDS data and topics, such as fault data from sources such as BMS and motors in **rt/lowstate(LowState_.idl)**, and faults in peripheral devices such as radar and UWB also have corresponding topics and idl. If you only care about the fault data of a specific device, you can directly obtain it using the corresponding topic and idl. However, if you need to subscribe to all faults, you can subscribe to the fault set. The fault set is obtained by the WebRTC module by subscribing to all fault related topics and integrating them.



A typical fault code is formatted as uint32, uint16, or uint8, with each bit representing a fault. A single fault code can represent up to 32, 16, or 8 faults. In order to distinguish different fault sources, the concept of fault source code was introduced.

Fault source code: UINT32	Fault code: UINT32 (maximum of 32 faults)

Topic: rt/errorlist, IDL: String_.idl

```cpp
#ifndef __std_msgs__msg__string__idl__
#define __std_msgs__msg__string__idl__

module std_msgs {
module msg {
module dds_ {
struct String_ {
string data;
};

};  // module dds_
};  // module msg
};  // module std_msgs

#endif  // __std_msgs__msg__string__idl__
```

Field	Type	Description	value
data	string (json)	Current fault set	{"errors":[[Timestamp, fault source, fault code], [Timestamp, fault source, fault code], [Timestamp, fault source, fault code], [Timestamp, fault source, fault code]]}

The fault code is defined as follows:

Fault source code: UINT32	Fault code: UINT32 (maximum of 32 faults)

Distinguish the fault source through the fault source code, and determine the specific fault for the same fault source through the fault code.

A single fault is implemented through bit operations, and each fault code represents up to 32 independent faults.

Error code

	Fault Source Code	Fault Source Topic	Fault IDL	Fault Code	Error Name	Other	Level
Bottom communication firmware	100	rt/lf/lowstate	LowState.bit_flag	0x20	Remote control communication error	Abandoned	Fault
				0x01	Bottom communication detection motion control DDS message timeout	Obsolete	Fault
					MCU communica		

Helpful
Feedback

				0x40	MCU communication abnormality		Fault
				0x80	Abnormal motor communication		Fault
				0x10	Abnormal battery communication		Fault
				0x02	Abnormal distribution switch		Fault
	200	rt/lf/lowstate	LowState.fanFrequency[3]	0x01	Left rear fan stuck	Abandoned	Fault
				0x02	Right rear fan blocked rotation		Fault
				0x04	Front fan blocked rotation		Fault
Motor	300+(leg*3 + motor)	rt/lf/lowstate	MotorState[leg*3 + motor].reserve[0]	0x01	Overcurrent	MotorState[leg*3 + motor].reserve[1] (communication frequency of this motor at this moment)	Fault
				0x02	Overvoltage fault		Fault
				0x04	Drive overheating		Fault
				0x08	Bus undervoltage		Fault
				0x10	Winding overheating		Fault
				0x20	Encoder abnormal		Fault
				0x100	Motor communication interrupted		Fault
Radar	400	rt/utlidar/lidar_state	LidarState.error_state	0x01	Abnormal motor speed		Fault
				0x02	Point cloud data abnormality		Fault
				0x04	Serial port data abnormality		Fault
UWB	500	rt/uwbstate	UwbState.error_state	0x01	UWB serial port opening exception	When abnormal, no UWB data	Fault
				0x02	Abnormal acquisition of robot status information	When abnormal, no UWB data	Fault
Motion Control							

The PDF has been downloaded. Rate us: ☆☆☆☆☆